



Programación Concurrente

Actividad 11 **“Redes de Petri”**

ALCANCE DEL TRABAJO

Redactar un documento (PDF o Word) con las respuestas a los ejercicios planteados. Se espera que las respuestas estén redactadas con palabras propias y demuestren comprensión, no solo copia de definiciones.

Objetivos

Al finalizar esta práctica el estudiante será capaz de:

- Comprender los fundamentos de las Redes de Petri como modelo formal para sistemas concurrentes.
- Modelar y analizar problemas clásicos de concurrencia mediante Redes de Petri.
- Verificar propiedades de correctitud como Safety, Liveness, Reachability y Boundedness.
- Relacionar el modelado formal con los mecanismos de sincronización y comunicación estudiados en los módulos anteriores.
- Implementar en Rust simulaciones que reflejen el comportamiento de Redes de Petri y validar su correspondencia con sistemas concurrentes reales.
- Integrar los conocimientos adquiridos en los módulos anteriores, comprendiendo cómo las Redes de Petri permiten modelar, analizar y verificar formalmente la correctitud de sistemas concurrentes y distribuidos antes de su implementación.

PARTE 1 — FUNDAMENTOS DEL MODELADO FORMAL

EJERCICIO 1 — ¿Por qué necesitamos Redes de Petri?

Hasta el momento estudiamos diferentes mecanismos de concurrencia e implementamos soluciones sincronizadas utilizando Rust.

Respondé:

- a) ¿Cuáles son las limitaciones fundamentales de verificar la correctitud de un sistema concurrente o distribuido únicamente mediante pruebas (*testing* empírico)?
- b) Explicá qué se entiende por **explosión combinatoria** del espacio de estados ($O(M^N)$).
- c) ¿Qué son los **Heisenbugs** y por qué aparecen frecuentemente en sistemas concurrentes?
- d) Explicá cómo el modelado formal ayuda a garantizar las propiedades de **Safety** y **Liveness** antes de implementar el código.

EJERCICIO 2 — Anatomía de una Red de Petri y Semántica Dinámica

Considerá una Red de Petri Marcada ($N = (P, T, F, W, M_0)$).

Respondé:

- a) ¿Qué representa cada uno de estos elementos en el modelado de un sistema real?
 - Lugar (P)
 - Transición (T)
 - Token / Marca
 - Arco (F) y Peso (W)

- b) Explicá cuándo una transición t se encuentra **habilitada** e indicá la condición matemática correspondiente vista en clase:

$$M(p) \geq W(p, t) \quad \forall p \in \bullet t$$

- c) Describí el **disparo** de una transición utilizando la ecuación algebraica de transición de estado:

$$M' = M - Pre + Post$$

- d) ¿Por qué se considera que el disparo de una transición es una operación **atómica** y sin tiempo implícito?
- e) Relacioná estos elementos formales con:
 - Mutex
 - Semáforos
 - Variables de estado / Condición de guardia

PARTE 2 — REDES DE PETRI Y LOS MÓDULOS ANTERIORES

EJERCICIO 3 — Exclusión Mutua

Observá la Red de Petri correspondiente al comportamiento de un Mutex con dos procesos compitiendo (*diagrama adjunto en clase*).

Respondé:

- a) ¿Qué representa el token ubicado en el lugar `Mutex` ?
- b) Explicá, **utilizando la estructura de la Red de Petri**, por qué solamente un proceso puede ingresar simultáneamente a la sección crítica.
- c) Explicá cómo la estructura de la red garantiza la propiedad de **Safety** ($M(P_{1_CS}) + M(P_{2_CS}) \leq 1$).
- d) Relacioná esta representación formal con la semántica del tipo `Arc<Mutex<T>>` y el guardián `RAII` en Rust.

EJERCICIO 4 — Productor–Consumidor

Considerá la Red de Petri del problema Productor–Consumidor con un buffer acotado de capacidad K .

Respondé:

- a) Identificá en la topología de la red:
 - Productor y Consumidor
 - Buffer
 - Espacios libres ($P_{vacíos}$)
 - Espacios ocupados (P_{llenos})
- b) Explicá cómo la red evita estructuralmente los problemas de:
 - **Overflow** (producir en buffer lleno)
 - **Underflow** (consumir de buffer vacío)
- c) ¿Qué propiedad (Invariante de Lugar) permanece constante durante toda la ejecución del sistema?
- d) Relacioná esta red con el problema clásico estudiado en los módulos anteriores.

EJERCICIO 5 — Modelo de Actores

Observá una Red de Petri que representa dos Actores aislados comunicándose mediante un buzón de mensajes.

Respondé:

- a) ¿Qué representa el lugar intermedio $P_{mailbox}$?
- b) ¿Qué representan los tokens circulando por dicho lugar?
- c) Explicá por qué la topología de la red demuestra que este modelo evita las **Data Races** por construcción.
- d) Relacioná este comportamiento con el uso de canales asincrónicos (`mpsc`) en Rust.

EJERCICIO 6 — Two-Phase Commit (2PC)

Considerá la Red de Petri que modela el protocolo de consenso distribuido Two-Phase Commit (2PC).

Respondé:

- a) Identificá los lugares y transiciones que corresponden a:
 - Coordinador
 - Participantes
 - Estado `Prepared`
 - Decisiones `Commit` y `Abort`
- b) Explicá qué ocurre con la distribución de tokens cuando el Coordinador falla (crash) tras la fase de votación.
- c) ¿Cómo representa la Red de Petri la ocurrencia de un **Deadlock Estructural** en este escenario?
- d) ¿Qué propiedad de correctitud (**Safety**) se mantiene inalterada incluso durante la falla del nodo?

PARTE 3 — VERIFICACIÓN Y ANÁLISIS DE LA RED

EJERCICIO 7 — Análisis de Propiedades Formales

Para cada una de las siguientes 5 propiedades de análisis de Redes de Petri, indicá qué significa, qué problema detecta y un ejemplo práctico:

- **Reachability** (Alcanzabilidad)
- **Boundedness** (Limitación / Acotamiento)
- **Safety** (Seguridad / 1-Boundedness)
- **Liveness** (Vivacidad)
- **Reversibility** (Reversibilidad)

Integración de criterio de ingeniería:

- **f)** Indicá cuál de estas propiedades considerarás la más crítica de verificar en cada uno de los siguientes sistemas y justificá brevemente:
 - Sistema bancario
 - Sistema ferroviario de interbloqueo
 - Plataforma IoT masiva
 - Servidor Web de alta concurrencia

EJERCICIO 8 — Análisis Dinámico y Traza Manual de una Red

Dada la Red de Petri del Productor–Consumidor vista en la diapositiva con un buffer de capacidad $K = 2$:

Respondé:

- **a)** Indicá el vector del marcado inicial M_0 .
- **b)** Enumerá cuáles transiciones se encuentran habilitadas en el estado M_0 .
- **c)** Ejecutá manualmente dos disparos consecutivos a elección, mostrando el vector de marcado intermedio M_1 y el resultante M_2 .
- **d)** ¿Qué transiciones quedan habilitadas luego de cada uno de los disparos?
- **e)** ¿Puede producirse un estado de Deadlock en esta red? Justificá utilizando el concepto de estado sin transiciones habilitadas.

EJERCICIO 9 — Redes de Petri Coloreadas (CPN)

Respondé:

- **a)** ¿Qué limitación de escalabilidad presentan las Redes de Petri clásicas cuando los datos son complejos?

- **b)** ¿Qué elementos nuevos incorporan las Redes de Petri Coloreadas (CPN)?
- **c)** Explicá cómo permiten modelar sistemas del mundo real sin sufrir la explosión visual del grafo.
- **d)** Relacioná los tokens coloreados y las guardas de las CPN con:
 - `struct` y `enum`
 - Cláusulas `match` e `if guard`
 - El concepto de **Ownership** en Rust.

PARTE 4 — IMPLEMENTACIÓN Y SIMULACIÓN EN RUST

EJERCICIO 10 — Motor de una Red de Petri y Exploración de Estados

Implementá en Rust una estructura `PetriNet` que permita:

- Almacenar el vector de marcado actual (`marking`).
- Representar las condiciones de entrada y salida mediante matrices de incidencia (`pre_matrix` y `post_matrix`).
- Verificar si una transición está habilitada (`is_enabled`).
- Disparar una transición de manera atómica actualizando el marcado (`fire`).
- **Explorar el espacio de estados:** Implementá un método `reachable_markings(&self)` `-> Vec<Vec<usize>>` que, mediante un algoritmo de búsqueda (BFS o DFS), explore y devuelva todos los marcados alcanzables desde el estado actual para una red pequeña.

```

1  use std::collections::HashSet;
2
3  struct PetriNet {
4      marking: Vec<usize>,
5      pre_matrix: Vec<Vec<usize>>, // pre_matrix[transicion][lugar]
6      post_matrix: Vec<Vec<usize>>, // post_matrix[transicion][lugar]
7  }
8
9  impl PetriNet {
10     fn new(initial_marking: Vec<usize>, pre: Vec<Vec<usize>>, post: Vec<Vec<usize>>) -> Self {
11         // ...
12     }
13
14     fn is_enabled(&self, transition: usize) -> bool {
15         // M(p) >= Pre(t, p) para todo p
16     }
17
18     fn fire(&mut self, transition: usize) -> Result<(), &'static str> {
19         // M'(p) = M(p) - Pre(t, p) + Post(t, p)
20     }
21
22     fn reachable_markings(&self) -> Vec<Vec<usize>> {
23         // Algoritmo para explorar los estados alcanzables R(M0)
24     }
25 }

```

Tarea: Utilizá tu implementación para simular la Red de Petri del Mutex (Ejercicio 3), ejecutá `reachable_markings()` y comprobá que en ningún marcado alcanzable existan dos procesos simultáneamente en sección crítica.

EJERCICIO 11 — Productor–Consumidor Asíncrono con Tokio

Implementá en Rust un sistema Productor–Consumidor utilizando un canal acotado

`tokio::sync::mpsc::channel(CAPACIDAD).`

Analizá y respondé en los comentarios del código:

- **a)** ¿En qué momento exacto la tarea Productora queda suspendida asincrónicamente?
- **b)** ¿Qué acción del Consumidor habilita nuevamente la ejecución del Productor?
- **c)** Explicá cómo se interpreta este comportamiento dinámico mediante el intercambio de tokens (P_vacíos y P_llenos) en la Red de Petri equivalente.

EJERCICIO 12 — Simulación de Deadlock por Timeout (2PC)

Implementá en Rust con Tokio una simulación del protocolo Two-Phase Commit donde se simule la caída del Coordinador mediante el cierre abrupto de un canal `oneshot` o un `tokio::time::timeout`.

Al ejecutar la simulación, respondé:

- **a)** ¿En qué estado del ciclo de vida queda detenido el proceso Participante?
- **b)** ¿Por qué la tarea no puede avanzar ni finalizar libremente?
- **c)** ¿Cómo se representa este estado de bloqueo indefinido en el Grafo de Alcanzabilidad de una Red de Petri?
- **d)** ¿Qué propiedad fundamental del sistema se preserva a pesar de que el proceso quedó bloqueado?

PARTE 5 — ANÁLISIS COMPARATIVO

EJERCICIO 13 — Selección del Modelo de Verificación Adecuado

Para cada uno de los siguientes escenarios, seleccioná la herramienta más adecuada (**FSM**, **Redes de Petri / CPN**, o **Testing Empírico**) y **justificá obligatoriamente** la decisión técnica considerando: Safety, Liveness, Concurrencia, Complejidad y Capacidad de Detección Previa.

Escenario de Software / Hardware	Modelo Seleccionado	Justificación Técnica (Obligatoria)
1. Formulario Web Secuencial (Multi-step UI)		
2. Pipeline Productor–Consumidor en Memoria		
3. Core de Transacciones Bancarias Concurrentes		
4. Protocolo de Consenso Distribuido (2PC / Raft)		

5. API REST CRUD convencional sobre Base de Datos		
6. Sistema de Interbloqueo de Control Ferroviario		

ENTREGA

La entrega deberá incluir:

- Resolución teórica de todos los ejercicios.
- Código fuente Rust de los ejercicios de implementación.
- Justificación técnica de cada decisión de diseño.
- Explicación de las propiedades de Safety y Liveness involucradas en cada caso.

Bibliografía recomendada:

- Andrews – Foundations of Multithreaded, Parallel and Distributed Programming.
- Ben-Ari – Principles of Concurrent and Distributed Programming.
- The Rust Programming Language.